Universitatea
Transilvania
din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

PROIECT SEMESTRUL II

Conducător științific:

Student:
PUIU Alin-George

BRAȘOV, 2025

Departamentul de Electronică și calculatoare
Programul de studii: Electronică Aplicată

Puiu Alin-George

RoBOBOt

Conducător științific:

Brașov, 2025

Cuprins

Cuprins

1 Introducere 4
1.1 Temă 4
1.2 Schema bloc a circuitului 5
1.3 Componente 5
1.3.1 Placa dezvoltare ESP32-WROOM-32S x1 5
1.3.2 Servo motoare SG90 x2 5
1.3.3 Buzzer-e pasive x2 5
1.3.4 Led rosu x1 5
1.3.5 Buton capacitiv TTP223 x1 5
1.3.6 Display OLED 0.96” 5
1.3.7 Corp robot x1 5
1.3.8 Cablu de date USB 5
2 Asamblare componente pe corpul robotului 6
2.1 Descrierea mod asamblare 6
2.2 Asamblare componente electronice 6
2.3 Descriere componente 7
2.3.1 Placa dezvoltare ESP32-WROOM-32S 7
2.3.2 Servo motor SG90 7
2.3.3 Buzzer 7
2.3.4 Buton capacitiv TTP223 7
2.3.5 Display OLED 0.96” 8
3 Descriere cod si functionalitati 8
3.1 Includerea bibliotecilor 8
3.2 Functii ecran oled 8
3.3 Functii sunet buzzer 8
3.4 Functii servo motoare 9
3.5 Starile robotului si accesarea lor 9
3.5.1 Functiile starilor si descrierea lor 9
3.5.2 Starea awake 10
3.5.3 Cod stari 11
3.5.4 Diagrama de stari 13
4 Bibliografie 14

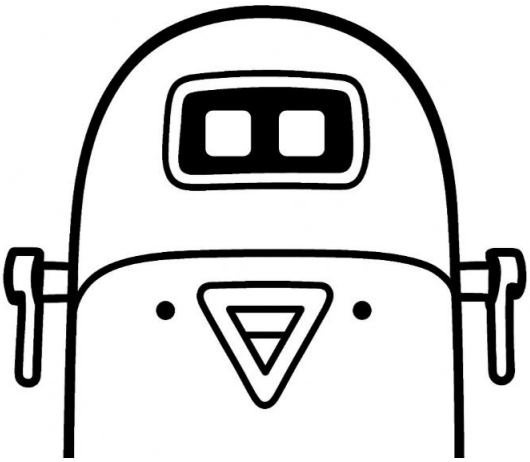
1 INTRODUCERE

1.1 Temă

Proiectul „RoBOBOt” reprezinta un asistent, un pet interactiv, robotizat de dimenisiuni mici. Acest mini roboțel este conceput să fie un companion interactiv, ideal pentru persoanele singure sau pentru copii care își doresc o prezență prietenoasă în viața lor. Dotat cu senzori tactili, roboțelul reacționează la atingere prin mișcări, sunete și expresii luminoase, creând senzația unei comunicări reale. Prin aceste reacții, el poate transmite empatie și afecțiune, devenind o sursă de confort emoțional și companie.

Roboțelul poate fi programat să recunoască diferite tipuri de atingeri — o mângâiere blândă poate declanșa o mișcare lentă și un sunet calmant, în timp ce o atingere mai energică poate stârni o reacție jucăușă și veselă. Astfel, el încurajează interacțiunea și stimulează simțurile, ajutând utilizatorul să se simtă mai conectat și mai puțin singur.

Pe lângă funcția de companion, roboțelul poate fi un excelent instrument pentru dezvoltarea emoțională a copiilor, învățându-i să recunoască și să exprime emoții printr-un partener de joacă non-judecător. Pentru adulți, poate oferi un sentiment de prezență și calm, reducând senzația de izolare.



1.2 SCHEMA BLOC A CIRCUITULUI

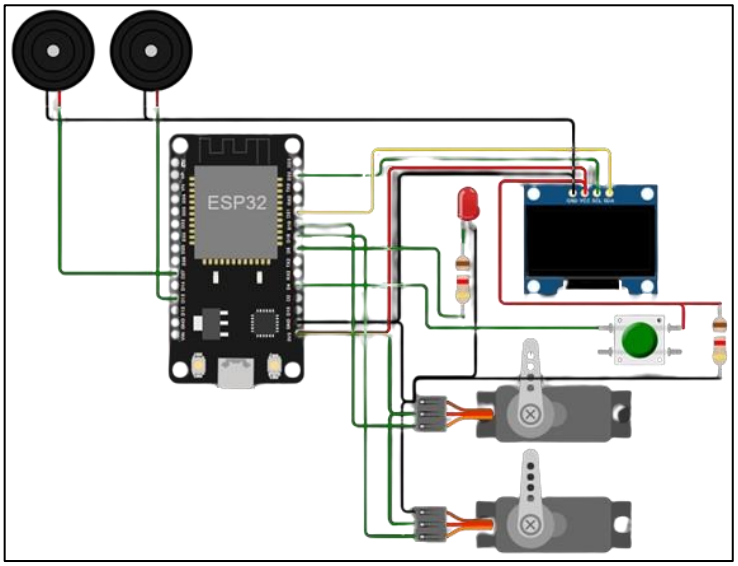


Figura 1. Schema bloc a circuitului

1.3 COMPONENTE

1.3.1 Placa dezvoltare ESP32-WROOM-32S x1

1.3.2 Servo motoare SG90 x2

1.3.3 Buzzer-e pasive x2

1.3.4 Led rosu x1

1.3.5 Buton capacitiv TTP223 x1

1.3.6 Display OLED 0.96”

1.3.7 Corp robot x1

1.3.8 Cablu de date USB

2 ASAMBLARE COMPONENTE PE CORPUL ROBOTULUI

2.1 DESCRIEREA MOD ASAMBLARE

Mini roboțelul este alcătuit din două părți principale Prima parte este o cutie în formă de paralelipiped, care servește drept bază stabilă pentru întregul sistem. În interiorul acesteia se află componentele esențiale pentru funcționarea robotului: o placă de dezvoltare, două buzzere montate pe o placă de prototipaj și un condensator de 100 µF. Buzzerele au rolul de a emite sunete în diverse momente ale interacțiunii, în timp ce condensatorul este utilizat pentru a filtra zgomotele electrice generate de cele două servo motoare, asigurând astfel o funcționare mai stabilă și mai curată din punct de vedere electric.

Cea de-a doua parte a roboțelului reprezintă corpul său propriu-zis, care include elementele de interfață și interacțiune cu utilizatorul. Acesta conține un ecran OLED, pe care pot fi afișate expresii faciale sau mesaje, un senzor capacitiv ce detectează atingerile, un LED pentru semnalizare vizuală și un servo motor care, prin intermediul unei biele, pune în mișcare brațele robotului, oferindu-i un plus de expresivitate.
Cele două părți sunt conectate printr-un al doilea servo motor, care permite corpului robotului să se rotească față de bază.

Figura 2. Ansamblu robot+componente montat

2.2 ASAMBLARE COMPONENTE ELECTRONICE

Asamblarea componentelor electronice pe corpul robotului s-a realizat prin lipire cu adezivi si prin prinderea cu suruburi.

Conexiunile dintre placa-mama si restul perifericelor s-au efectuat cu ajutorul unor fire cu o sectiune de 0.33mm^2 lipite cu la cald cu fludor de fiecare componenta.

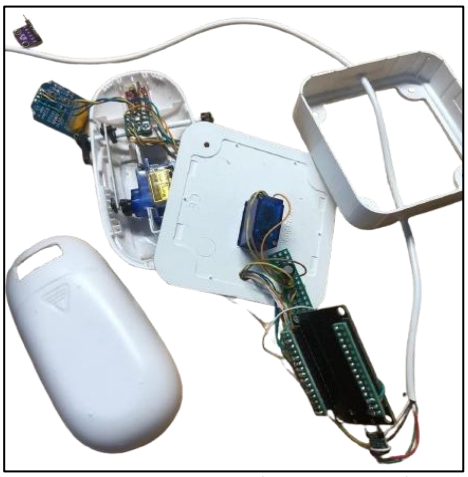


Figura 3. Conexiunile componentelor

2.3 DESCRIERE COMPONENTE

2.3.1 Placa dezvoltare ESP32-WROOM-32S

Modulul ESP32-WROOM-32S este o placă de dezvoltare bazată pe sistemul pe cip (SoC) ESP32-D0WDQ6 de la Espressif Systems, integrat într-un format SMD cu memorie Flash SPI de 4 MB. Acesta include un procesor dual-core Xtensa® 32-bit LX6, tactat la frecvențe de până la 240 MHz, capabil să execute aplicații în timp real și multitasking pe nucleele separate.

2.3.2 Servo motor SG90

Servo motorul SG90 este un microservo compact și ușor, frecvent utilizat în aplicații de robotică, modelism și sisteme embedded care necesită poziționare unghiulară precisă. Acesta funcționează pe baza unui motor de curent continuu cu reductor intern și sistem de feedback, permițând controlul poziției axului într-un interval de aproximativ 0° până la 180°. SG90 este alimentat în general cu o tensiune cuprinsă între 4.8 V și 6 V și este controlat prin semnal PWM, cu o frecvență standard de 50 Hz și lățimi de impuls între 1 ms și 2 ms, corespunzătoare capetelor de cursă

2.3.3 Buzzer

Buzzer-ele pasive sunt componente electroacustice utilizate pentru generarea de semnale sonore, fără a conține un generator intern de frecvență. Ele necesită aplicarea unui semnal electric de tip PWM pentru a produce vibrații ale membranei interne, ceea ce generează sunetul

2.3.4 Buton capacitiv TTP223

Butonul capacitiv TTP223 este un senzor tactil bazat pe un circuit integrat care detectează atingerea prin variații de capacitanță, fără a necesita contact mecanic.

2.3.5 Display OLED 0.96”

Display-ul OLED de 0.96” este un ecran grafic de mici dimensiuni, bazat pe tehnologia OLED (Organic Light-Emitting Diode), care oferă un contrast ridicat, unghiuri largi de vizualizare și consum redus de energie. Acest tip de afișaj nu necesită iluminare de fundal, fiecare pixel emițând propria lumină, ceea ce permite obținerea unui negru profund și a unei claritați excelente a caracterelor afișate.

3 DESCRIERE COD SI FUNCTIONALITATI

3.1 INCLUDEREA BIBLIOTECILOR

```
#include <ESP32Servo.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

Acest fragment de cod include biblioteci necesare pentru controlul unui servomotor și afișarea de informații pe un ecran OLED, folosind o placă ESP32. Biblioteca ESP32Servo permite controlul precis al servomotoarelor, Wire gestionează comunicația I2C, iar Adafruit_GFX împreună cu Adafruit_SSD1306 oferă suport pentru afișarea grafică pe ecranele OLED SSD1306.

3.2 FUNCTII ECRAN OLED

```
void angry_eyes() {
  display.clearDisplay();
  int tri_width = 40, tri_height = 35, center_y = 32;
  display.fillTriangle(21, center_y + tri_height / 2, 21, center_y - tri_height / 2, 21 +
tri_width, center_y + tri_height / 2, SSD1306_WHITE);
  display.fillTriangle(107, center_y + tri_height / 2, 107, center_y - tri_height / 2, 107 -
tri_width, center_y + tri_height / 2, SSD1306_WHITE);
  display.display();
}
```

display.fillTriangle(x0, y0, x1, y1, x2, y2, color) desenează un triunghi umplut cu culoarea specificată, folosind cele trei puncte date ca vârfuri. În contextul funcției angry_eyes(), este folosită pentru a desena doi ochi simetrici cu ipotenuza spre centru care dau expresia „mânioasă” feței.

3.3 FUNCTII SUNET BUZZER

```
void playBeep(int freq, int duration) {
  tone(BUZZER_PIN, freq); delay(duration); noTone(BUZZER_PIN);
}
```

```
void playRandomBeeps(int count) {
  for (int i = 0; i < count; i++) {
    playBeep(random(300, 1200), random(80, 150));
    delay(random(50, 150));
  }
}
```

Funcția playBeep() redă un sunet de o frecvență și durată specificate folosind un buzzer.Funcția playRandomBeeps() redă o serie de sunete (număr specificat prin count) cu frecvențe și durate aleatorii, creând un efect sonor variat și haotic.

3.4 FUNCTII SERVO MOTOARE

```
void servo1Task(void *pvParameters) {
  int currentPos = 90;
  servo1.write(currentPos);
  while(true) {
    portENTER_CRITICAL(&mux);
    int target = servo1Target;
    portEXIT_CRITICAL(&mux);

    if(currentPos < target) {
      currentPos+=1;
      servo1.write(currentPos);
      vTaskDelay(10 / portTICK_PERIOD_MS);
    } else if(currentPos > target) {
      currentPos-=1;
      servo1.write(currentPos);
      vTaskDelay(10 / portTICK_PERIOD_MS);
    } else {
      vTaskDelay(30 / portTICK_PERIOD_MS);
    }
  }
}
```

Funcția servo1Task() controlează poziția unui servo motor într-un task FreeRTOS.
Ea pornește de la poziția 90 și ajustează treptat poziția curentă (currentPos) pentru a ajunge la valoarea țintă servo1Target. Accesul la servo1Target este protejat cu un mutex (mux) pentru siguranța firelor de execuție. Mișcarea servo-ului este lină, cu pași de 1 grad și întârzieri scurte între pași. Dacă poziția dorită este atinsă, task-ul așteaptă un timp mai mare înainte de a verifica din nou.

3.5 STARILE ROBOTULUI SI ACCESAREA LOR

3.5.1 Functiile starilor si descrierea lor

Robotelul are 3 stari importante: starea de sleep, starea awake si starea angry. Starea de sleep este principala, sistemul pornind cu aceasta, la atingerea senzorului, intra in starea awake, in

starea awake la atingerea inca o data a senzorului, intra in starea angry, dupa un timp de inactivitate, intra in starea sleep.

3.5.2 Starea awake

```
void awake_state()
{
  blink();

   //Miscare la dreapta
  move_right_big_eye();

  portENTER_CRITICAL(&mux);
  servo1Target = 155;
  portEXIT_CRITICAL(&mux);
  vTaskDelay(2500 / portTICK_PERIOD_MS);

  reset_eyes_position_and_size();
  for(int pos = 0; pos <= 60; pos+=5) {
    portENTER_CRITICAL(&mux);
    servo2Target = pos;
    portEXIT_CRITICAL(&mux);
    vTaskDelay(20 / portTICK_PERIOD_MS);
  }
  playMelody1();


  // Revenire
  portENTER_CRITICAL(&mux);
  servo1Target = 90;
  servo2Target = 0;
  portEXIT_CRITICAL(&mux);
  vTaskDelay(20 / portTICK_PERIOD_MS);

  // Miscare la stanga
  move_left_big_eye();

  portENTER_CRITICAL(&mux);
  servo1Target = 25;
  portEXIT_CRITICAL(&mux);
    vTaskDelay(2500 / portTICK_PERIOD_MS);

  reset_eyes_position_and_size();
  for(int pos = 0; pos <= 60; pos+=5) {
    portENTER_CRITICAL(&mux);
    servo2Target = pos;
    portEXIT_CRITICAL(&mux);
    vTaskDelay(20 / portTICK_PERIOD_MS);
  }
```

```
  playMelody1();

  portENTER_CRITICAL(&mux);
  servo1Target = 90;
  servo2Target = 0;
  portEXIT_CRITICAL(&mux);
  reset_eyes_position_and_size();
  vTaskDelay(20 / portTICK_PERIOD_MS);
}
```

In starea awake robotelul isi indreapta privirea spre o directie, indrepatandu-se spre aceasta, miscandu-si tot o data si bratele, cu efecte sonore

3.5.3 Cod stari

```
void loop() {
  static bool lastButtonState = LOW;
  bool currentButtonState = digitalRead(BUTTON_PIN);

  // Detectare apăsare (rising edge: LOW -> HIGH)
  if (lastButtonState == LOW && currentButtonState == HIGH) {
    switch (currentState) {
      case STATE_SLEEPING:
        currentState = STATE_WAKEUP;
        Serial.println("State: WAKEUP");
        lastAwakeTimestamp = millis();
        break;

      case STATE_BLINK:
        currentState = STATE_ANGRY;
        Serial.println("State: ANGRY");
        break;

      default:
        // Alte stări nu necesită acțiune directă la apăsare
        break;
    }
  }
  // Comportament în funcție de starea curentă
  switch (currentState) {
    case STATE_SLEEPING:
      sleeping_state();
      break;

    case STATE_WAKEUP:
      wakeup_state();
      currentState = STATE_AWAKE;
      lastAwakeTimestamp = millis();
```

```
      break;

    case STATE_BLINK: {
  static unsigned long lastBlinkTime = 0;
  unsigned long now = millis();
  const unsigned long blinkInterval = 2000; // interval blink mai rar (2 sec)

  if (now - lastBlinkTime >= blinkInterval) {
    blink();
    lastBlinkTime = now;
  }

  if (now - lastAwakeTimestamp > 20000) {
    currentState = STATE_FALLING_ASLEEP;
    Serial.println("State: FALLING ASLEEP");
  }
  break;
}

    case STATE_AWAKE:
      awake_state();
      reset_eyes_position_and_size();
      currentState = STATE_BLINK;
      // După 20s fără acțiune → adormire

      break;

    case STATE_ANGRY:
      angry_state();
      currentState = STATE_BLINK;
      lastAwakeTimestamp = millis();
      break;

    case STATE_FALLING_ASLEEP:

      fallingasleep_state();
      currentState = STATE_SLEEPING;
      break;
  }
lastButtonState = currentButtonState;
  vTaskDelay(10 / portTICK_PERIOD_MS); // Pauză mică pentru multitasking
}
```

3.5.4 Diagrama de stari

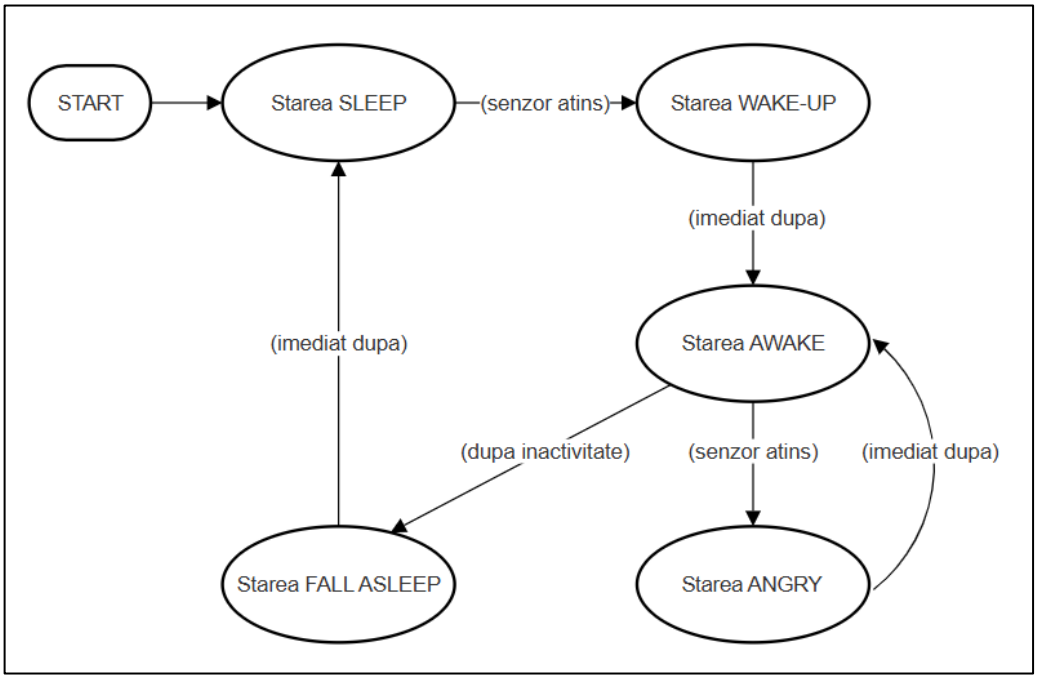


Figura 4. Diagrama stari

4 BIBLIOGRAFIE

[1] Afisare ochi: https://youtu.be/bOCKteaR9Z4?si=TWs8c7Xtmz0RmdMq

[2] Implementare cod arduino pentru ESP32:https://forum.arduino.cc/t/freertos-example-codes-for-esp32/682084